LVM at linux, filesystem, shrink/resize , volume group,

Wednesday, March 18, 2026
3:29 PM

Βήμα 1 -> Adding Hard disks

After I added 3 hard disks the each one of 1.00 GB
When I run the `lsblk` command I can see that the hard disk where are added.

And specifically, it recognises each physical volume as the sdb, sdc & sdd!!

The main disks are named as the sda.
Lvm = logical volume manager

Logical Volume Manager (LVM)

is used on Linux to manage hard drives and other storage devices. As the name implies, it can sort raw storage into logical volumes, making it easy to configure and use.

```
-bash: lsbk: command not found
[root@L9mpip ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                   8:0    0   20G  0 disk
├─sda1                8:1    0    1G  0 part /boot
└─sda2                8:2    0   19G  0 part
  ├─ol_l9mpip-root 252:0    0   17G  0 lvm  /
  └─ol_l9mpip-swap 252:1    0    2G  0 lvm  [SWAP]
sdb                   8:16   0    1G  0 disk
sdc                   8:32   0    1G  0 disk
sdd                   8:48   0    1G  0 disk
sr0                  11:0    1 1024M  0 rom
[root@L9mpip ~]#
```

The steps of that project are:

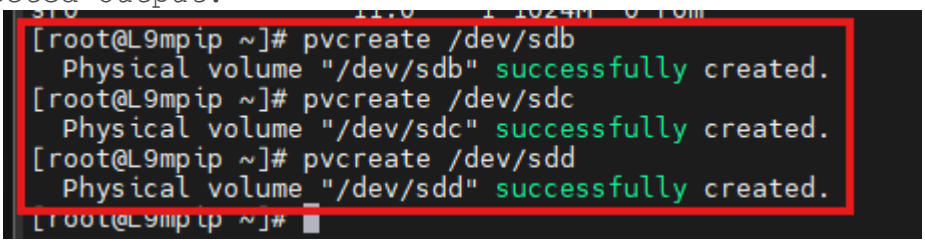
- How to create partitions
- How to create physical volumes
- How to create a virtual group
- How to create logical volumes
- How to create a filesystem on logical volumes
- How to mount logical volumes
- How to extend a logical volume
- How to remove a logical volume

Βήμα 2 -> Create physical volumes

We can now create physical volumes on our new partitions by using the **`pvcreate`** command.

```
# pvcreate /dev/sdb /dev/sdc /dev/sdd
```

Expected Output:



Commands :

- Lblk = list type of storage

- # Physical volume
- Pvcreate
- Pvdisplay
- Pvs

#Volume group
- Vgcreate
- Vgdisplay
- Vgs

Logical group
- Lgcreate
- Lgdisplay
- Lgs

#Filesystem

- Mkfs.ext4

Then I should do a mount

Listing now the physical volumes :

```
[root@L9mpip ~]# pvs
  PV         VG        Fmt  Attr PSize   PFree
  /dev/sda2  ol_l9mpip lvm2 a--  <19.00g    0
  /dev/sdb             lvm2 ---    1.00g 1.00g
  /dev/sdc             lvm2 ---    1.00g 1.00g
  /dev/sdd             lvm2 ---    1.00g 1.00g
```

Use the `pvdisplay` command to see information about all the physical volumes on your system, or specify a particular volume you wish to view details about.

```
[root@L9mpip ~]# pvdisplay
  --- Physical volume ---
  PV Name               /dev/sdb
  VG Name               my_vg
  PV Size               1.00 GiB / not usable 4.00 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              255
  Free PE               255
  Allocated PE          0
  PV UUID               QVNbWo-ipBj-8oh7-nVnv-EPjR-1WET-c835ig

  --- Physical volume ---
  PV Name               /dev/sdc
  VG Name               my_vg
  PV Size               1.00 GiB / not usable 4.00 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              255
  Free PE               255
  Allocated PE          0
  PV UUID               iar62H-jWqq-GWUA-zqjY-4oBb-9hs9-S8TlsC

  --- Physical volume ---
  PV Name               /dev/sdd
  VG Name               my_vg
  PV Size               1.00 GiB / not usable 4.00 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              255
  Free PE               255
  Allocated PE          0
  PV UUID               20pgwR-Z0e4-rgme-Geth-jkV0-vJ1V-rQJtU6

  --- Physical volume ---
  PV Name               /dev/sda2
  VG Name               ol_l9mpip
  PV Size               <19.00 GiB / not usable 3.00 MiB
  Allocatable           yes (but full)
  PE Size               4.00 MiB
  Total PE              4863
  Free PE               0
  Allocated PE          4863
  PV UUID               83h27F-RxOj-BDeD-DtX7-5YPb-tjWk-MQND0H
```

Βήμα 3 -> Create a virtual group

At this stage we need to create a virtual group which will serve as a container for our physical volumes. In this example, we'll call our virtual group "my_vg" which will include the **/dev/sdb1** partition, with the following :

```
[root@L9mpip ~]# sudo vgcreate my_vg /dev/sdb /dev/sdc /dev/sdd
  Volume group "my_vg" successfully created
```

Use the following command to display information about the virtual group.

```
# vgdisplay
```

```
[root@L9mpip ~]# vgdisplay
  --- Volume group ---
  VG Name               my_vg
  System ID
  Format                lvm2
  Metadata Areas        3
  Metadata Sequence No  1
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                0
  Open LV               0
  Max PV                0
  Cur PV                3
  Act PV                3
  VG Size               <2.99 GiB
  PE Size               4.00 MiB
  Total PE              765
  Alloc PE / Size       0 / 0
  Free  PE / Size       765 / <2.99 GiB
  VG UUID               SbJWuv-3aIf-oBts-yFf1-WFw2-swTg-Aa1S2l

  --- Volume group ---
  VG Name               ol_l9mpip
  System ID
  Format                lvm2
  Metadata Areas        1
  Metadata Sequence No  3
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                2
  Open LV               2
  Max PV                0
  Cur PV                1
  Act PV                1
  VG Size               <19.00 GiB
  PE Size               4.00 MiB
  Total PE              4863
  Alloc PE / Size       4863 / <19.00 GiB
  Free  PE / Size       0 / 0
  VG UUID               gpxq8Z-mR0q-p6Vs-RKZo-ijCe-SwTG-ggFfX5
```

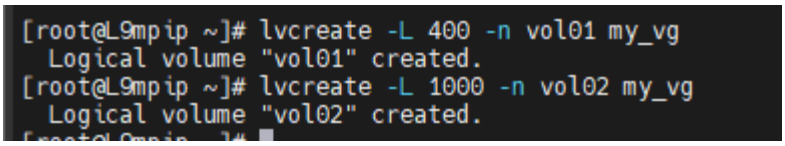
Βήμα 4 -> Create logical volumes

Now we can move on to creating logical volumes. It may help to think of our virtual group as a “big cake,” from which we can cut “pieces” (logical volumes) that will get treated as partitions on our Linux system.
The following command will create a logical volume named **`vol01`** with a size of 400MB.

```
# lvcreate -L 400 -n vol01 my_vg
```

Then, we’ll create another volume named **`vol02`** with a size of 1GB. Again, refer to the diagram above to help visualize the configuration.

```
# lvcreate -L 1000 -n vol02 my_vg
```



Finally, we can use the **`lvdisplay`** command to see the logical volumes we’ve just created.

```
[root@L9mpip ~]# lvdisplay
  --- Logical volume ---
  LV Path                /dev/my_vg/vol01
  LV Name                vol01
  VG Name                my_vg
  LV UUID                3DiR5r-oFNX-R9pb-Cw9I-tPDb-QdYQ-LHiFEU
  LV Write Access        read/write
  LV Creation host, time L9mpip, 2026-03-18 16:30:54 +0200
  LV Status              available
  # open                 0
  LV Size                400.00 MiB
  Current LE             100
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:2

  --- Logical volume ---
  LV Path                /dev/my_vg/vol02
  LV Name                vol02
  VG Name                my_vg
  LV UUID                Hk0NY7-7fuJ-iQqD-VGI0-kQSY-irRC-8AUbzS
  LV Write Access        read/write
  LV Creation host, time L9mpip, 2026-03-18 16:31:01 +0200
  LV Status              available
  # open                 0
  LV Size                1000.00 MiB
  Current LE             250
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:3
```

```
  --- Logical volume ---
  LV Path                /dev/ol_l9mpip/swap
  LV Name                swap
  VG Name                ol_l9mpip
  LV UUID                p6z4Kn-LowV-3K4c-z7Hj-4OkQ-QVEK-BuKkxY
  LV Write Access        read/write
  LV Creation host, time localhost.localdomain, 2025-11-20 10:50:22 +0200
  LV Status              available
  # open                 2
  LV Size                2.00 GiB
  Current LE             512
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:1

  --- Logical volume ---
  LV Path                /dev/ol_l9mpip/root
  LV Name                root
  VG Name                ol_l9mpip
  LV UUID                Ay5OPU-3tye-lixT-MY6C-3k1i-JRWL-rmxiy0
  LV Write Access        read/write
  LV Creation host, time localhost.localdomain, 2025-11-20 10:50:22 +0200
  LV Status              available
  # open                 1
  LV Size                <17.00 GiB
  Current LE             4351
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:0
```

Physical Volume (PV)

What: Raw storage devices prepared for LVM

- Physical disks, partitions, or RAID arrays
- Must be initialized with pvcreate

Visual Example

plaintext

```
Physical Disks:    [/dev/sdb] [/dev/sdc] [/dev/sdd]
                       ↓          ↓          ↓
Physical Volumes:   [PV1]      [PV2]      [PV3]
                       ↓__________↓__________↓
Volume Group:          [     my_vg (30GB)     ]
                       ↓          ↓          ↓
Logical Volumes:    [lv_root] [lv_home] [lv_nfs]
                       ↓          ↓          ↓
Filesystems:        [ext4]     [xfs]     [ext4]
```

As you can see from the screenshot below, `vgdisplay` shows us that we still have 1.6GB of free space in the `my_vg` virtual group.

```
[root@L9mpip ~]# vgdisplay
  --- Volume group ---
  VG Name               my_vg
  System ID
  Format                lvm2
  Metadata Areas        3
  Metadata Sequence No  3
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                2
  Open LV               0
  Max PV                0
  Cur PV                3
  Act PV                3
  VG Size               <2.99 GiB
  PE Size               4.00 MiB
  Total PE              765
  Alloc PE / Size       350 / <1.37 GiB
  Free  PE / Size       415 / 1.62 GiB
  VG UUID               SbJWuv-3aIf-oBts-yFf1-WFw2-swTg-Aa1S2l
```

Volume Group (VG)

What: Pool of storage combining one or more PVs

- Like a "storage pool" that aggregates physical storage
- You can add/remove PVs from a VG dynamically

Logical Volume (LV)

What: Virtual partitions carved from VG space

- Acts like traditional partitions but flexible
- Can be resized, moved, snapshotted
- Where you create filesystems

After creating the filesystem on the Logical Volume, you **mount it** to a directory

Βήμα 5 -> Create a filesystem on logical volumes

The logical volume is almost ready to use. All we need to do is to create a filesystem on it with the `mkfs` command.

```
# mkfs.ext4 -m 0 /dev/my_vg/vol01
```

The `-m` option specifies the percentage reserved for the super-user, we can set this to 0 to use all the available space (the default is 5%).

```
[root@L9mpip ~]# mkfs.ext4 -m 0 /dev/my_vg/vol01
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 409600 1k blocks and 102400 inodes
Filesystem UUID: 21eb6799-9b1f-49b5-beb6-dbbd420fd1ee
Superblock backups stored on blocks:
        8193, 24577, 40961, 57345, 73729, 204801, 221185, 401409

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

(creating an ext4 filesystem on the logical volume.)

Ext4 filesystem =

is a high-performance, open-source journaling file system used as the default in most modern Linux distributions.

BRAINSTORMING

What is a “mounted filesystem”?

A filesystem (like ext4, xfs) is just a way of organizing data on a disk or partition.
But in Linux, that filesystem is not usable by itself until it is attached to the directory tree.
That attachment is called a mount!!

Think of it like this

Imagine:

- The disk = a USB drive with files inside
- The filesystem = how those files are organized
- The mount = plugging it in and choosing where it appears

Until you mount it, it's like:

- "The drive exists, but you haven't opened it."

What happens before mounting?

You created a filesystem:

```
mkfs.ext4 /dev/my_vg/vol01
```

At this point:

- The filesystem exists
- But it is floating / unattached
- Linux doesn't show it in df
- You cannot access its files

What happens after mounting?

After:

```
mount /dev/my_vg/vol01 /mnt/vol01
```

Now:

- It becomes part of the system
- You can:

cd /mnt/vol01
touch file.txt
df shows it

Why do we need mounting?

1. To access data

Without mounting → filesystem is invisible to users and programs.

2. To control where storage appears

You decide:
/data
/backup
/var/log
This gives flexibility.

3. To manage multiple disks cleanly

You can attach many disks into one tree:

/
/home
/var
/data
Each could be a different filesystem.

4. To apply options and permissions

When mounting, you can control:
read-only vs read-write

security options
performance settings

Example:

mount -o ro /dev/sdb1 /mnt/test

Important concept

A filesystem can exist without being mounted
But it is useless until mounted

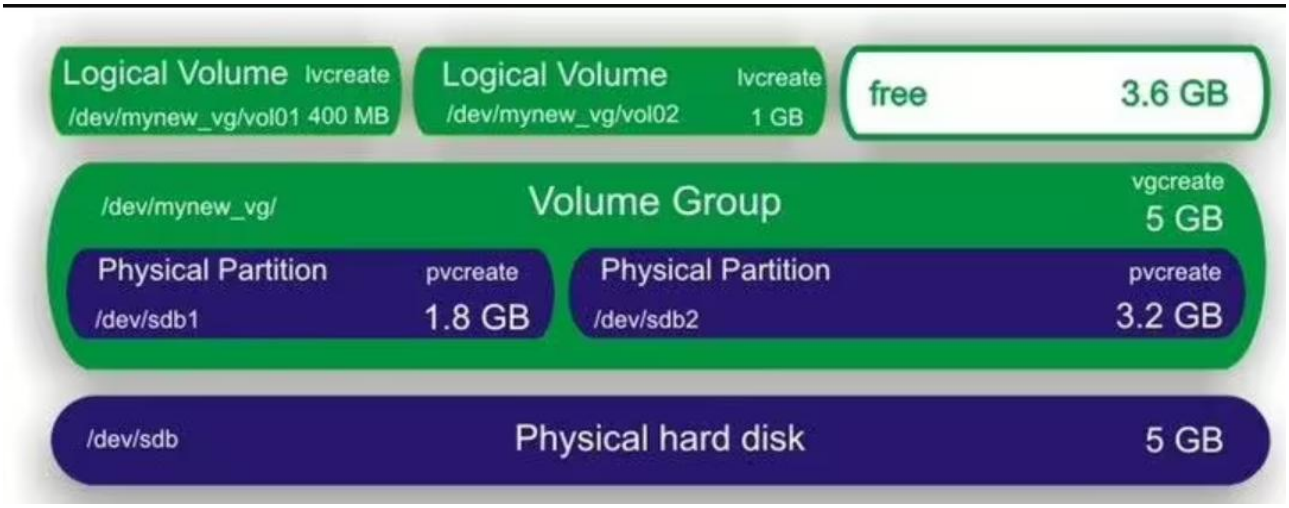
in order to verify it exist the filesystem

Run
lsblk -f

And after that section I need to mount it

Extra -> Extend a logical volume

The biggest advantage of a logical volume is that it can be extended any time we are running out of space. For example, to increase the size of a logical volume and add other 800 MB of space, we can run this command:

```
# lvextend -L +800 /dev/my_vg/vol01
```

Notice in the screenshot below that the command doesn't actually increase the size of the filesystem, but only that of the logical volume.